SWE 363 – Milestone 3 Requirements

Table of Contents

1. Motivation 1
2. Target Users 2
   a. Client 2
   b. Team Manager 2
   c. Team member 2
3. Requirement Modeling 2
   a. Functional Requirements 2
      i. Client's Functional Requirements 2
      ii. Team Manager's Functional Requirements 4
      iii. Team Member's Functional Requirements 6
   b. Non-Functional Requirements 9
      i. Quality Constraints 9
      ii. Security Constraints 9
      iii. Compatibility & Interface Constraints 10
      iv. Technical & External Constraints 10
4. Wireframe Development 10

1. Motivation

Many student projects and small team developments struggle with managing requirements effectively. Requirements are often written in scattered documents, chat messages, or spreadsheets, making it difficult to track changes, review updates, assign responsibilities, or maintain version history. As projects evolve, especially in agile environments where changes happen frequently, this lack of structure leads to confusion, duplicated work, miscommunication, and loss of traceability.

ReqFlow aims to solve this problem by providing a lightweight and focused requirements management platform tailored specifically for students and small teams. Instead of using complex enterprise tools that are overwhelming and unnecessary for small-scale projects, ReqFlow offers a simple, flexible, and structured environment to create, review, validate, and track requirements efficiently. By centralizing requirement management and supporting agile changes, the system helps teams stay organized, improve collaboration, and maintain clarity throughout the project lifecycle.

2. Target Users

a. Client

The Client represents the party requesting the system or project. This user is primarily responsible for expressing needs, submitting new requirements, and validating whether the implemented requirements meet expectations. The client interacts with the system mainly during the elicitation and validation stages by creating requirement requests, reviewing submitted requirements, providing feedback, and approving or rejecting finalized requirements.

The Client does not manage technical details or team assignments but plays a critical role in ensuring that the project aligns with business or academic objectives.

b. Team Manager

The Team Manager oversees the project's requirements management process. This user coordinates the workflow between clients and team members, ensures that requirements are properly analyzed and documented, and manages approval and validation cycles.

The Manager interacts with the system by reviewing requirements, assigning tasks to team members, resolving conflicts, managing requirement statuses, and maintaining overall project structure. The Manager has higher level permissions compared to other roles and ensures that requirements move correctly through their lifecycle.

c. Team member

The Team Member is responsible for analyzing, refining, and documenting requirements. This user works on transforming client requests into structured, clear, and actionable requirements.

Team Members interact with the system by editing requirement details, adding descriptions, defining acceptance criteria, linking related requirements, and participating in discussions. They do not approve of final requirements but contribute to preparing them for review and validation.

3. Requirement Modeling

a. Functional Requirements

i. Client's Functional Requirements

1. Create New Requirement

1. Client logs into the system using valid credentials.
2. Client selects the relevant project.

3. Client navigates to the “Create Requirement” page.
4. The system displays input fields (Title, Description).
5. Client enters requirement details.
6. Client clicks “Submit.”
7. The system validates required fields.
8. The system generates a unique requirement ID.
9. The system stores the requirement with status Draft and associates it with the Client.
10. A confirmation message is displayed.

2. Modify a Draft Requirement

1. Client navigates to “My Requirements.”
2. System displays submitted requirements with their status.
3. Client selects a requirement in Draft status.
4. Client clicks “Edit.”
5. System displays editable fields.
6. Client updates the content.
7. Client clicks “Save.”
8. System validates inputs.
9. System updates the requirement and records a new version entry.

If the requirement status is Review or beyond:

- The system disables editing and displays a restriction message.

3. Respond to Clarification Requests

1. Client opens a requirement.
2. Client navigates to the discussion/comments section.
3. Client reads clarification request.
4. Client enters response.
5. Client clicks “Post.”
6. System validates input.
7. System stores response in discussion thread.

4. Review Refined Requirement

1. Client navigates to requirements in “Under Review” status.
2. Client selects a requirement.
3. System displays the latest refined version.
4. System visually indicates changes between original and current version.
5. Client reviews the updated content.

5. Approve a Requirement

1. Client opens a requirement in “Under Review” status.
2. Client reviews content.
3. Client clicks “Approve.”
4. System requests confirmation.
5. Client confirms action.
6. System updates status to Approved.

6. Reject a Requirement with Justification

1. Client opens a requirement in “Under Review” status.
2. Client clicks “Reject.”
3. System prompts for justification text.
4. Client enters written feedback.
5. Client confirms rejection.
6. System updates status to Rejected and records feedback.

7. Track Requirement Status

1. Client navigates to “My Requirements.”
2. System displays list of submitted requirements.
3. Each requirement shows:
    - Requirement ID
    - Title
    - Current status (Draft, Review, Approved, Rejected)
4. Client can sort or filter by status.

ii. Team Manager’s Functional Requirements

1. Manage Project Roles and Access Permissions

1. Manager navigates to 'Project Users'.
2. System displays all project users and their roles.
3. Manager selects a user and chooses a new role (Member or Manager).
4. System validates that the Client role is not modified and that at least one Manager remains in the project.
5. Manager confirms the change.
6. System updates permissions, stores the change, applies new access rights immediately, and logs the action with timestamp.

2. Define Workflow Stages

1. Manager opens Workflow settings.
2. System displays Draft, Review, Approved, and Rejected stages.
3. Manager adjusts visibility labels if needed.

4. System validates that Draft and Approved stages remain available.
5. Manager saves configuration.
6. System stores the workflow configuration and applies changes across the project.

3. Assign Requirement to Team Member

1. Manager opens a requirement.
2. Manager clicks Assign.
3. System displays a list of Team Members.
4. Manager selects a member.
5. System validates that the selected user has the Team Member role.
6. Manager confirms assignment.
7. System stores the assignment, updates the requirement view, generates a notification, and logs the action.

4. Reassign Requirement

1. Manager opens an assigned requirement.
2. Manager clicks Reassign.
3. System displays Team Members.
4. Manager selects a new member.
5. System validates that the new assignee is a valid Team Member.
6. Manager confirms.
7. System updates the assignment, notifies the new assignee, and logs the reassignment.

5. Set Deadline for Requirement

1. Manager opens a requirement.
2. Manager selects Set Deadline.
3. Manager chooses a date.
4. System validates that the selected date is valid.
5. Manager confirms.
6. System saves the deadline, displays it on the requirement, and logs the action.

6. Notify Manager When Deadline Exceeds

1. System periodically checks all requirements with deadlines.
2. If a deadline has passed and the requirement is not Approved or Locked, the system generates an overdue notification.
3. System stores the notification and displays an alert in the Manager dashboard.

7. Mark Requirements as Duplicates

1. Manager opens the project requirements list.
2. Manager selects two or more requirements.

3. System validates that at least two requirements are selected and that they belong to the same project.
4. Manager clicks Mark as duplicates.
5. System asks for confirmation.
6. Manager confirms.
7. System flags the requirements as duplicates, stores the relationship, and logs the action.

8. Merge Duplicate Requirements

1. Manager opens requirements marked as duplicates.
2. Manager clicks Merge duplicates.
3. System validates that the requirements are marked as duplicates.
4. System displays a merge view showing content of each requirement.
5. Manager selects the primary requirement to keep.
6. System validates that one primary requirement is selected.
7. Manager confirms the merge.
8. System merges content, archives duplicates, updates history, and logs the action.

9. Lock an Approved Requirement

1. Manager opens an Approved requirement.
2. Manager clicks Lock.
3. System validates that the requirement status is Approved.
4. System asks for confirmation.
5. Manager confirms.
6. System locks the requirement, disables editing for all roles, and records the lock event.

10. Display Overview Dashboard

1. Manager opens Dashboard.
2. System retrieves data limited to the selected project.
3. System computes total by status (Draft, Review, Approved, Rejected).
4. System highlights overdue requirements.
5. Manager may filter by status.
6. System updates the displayed list dynamically.

iii. Team Member's Functional Requirements

1. View Assigned Requirements

1. Team Member logs into the system.
2. Team Member navigates to 'My Assigned Requirements'.
3. System validates that only requirements assigned to that Team Member are retrieved.

4. System retrieves assigned requirements from the database.
5. System displays requirement ID, title, status, priority, and deadline (if defined).

2. Refine Requirement Description (Without Changing Intent)

1. Team Member opens an assigned requirement in Draft or Review stage.
2. System validates that the requirement is assigned to that Team Member and is not Approved and locked.
3. If clarification is needed, Team Member uses the clarification feature before editing.
4. Team Member clicks 'Edit Description'.
5. Team Member rewrites the requirement for clarity and precision.
6. Team Member provides a non-empty justification explaining what was changed and why.
7. Team Member clicks 'Save'.
8. System validates inputs, preserves the previous version, creates a new version entry, and logs editor identity with timestamp.

3. Classify Requirement Type

1. Team Member opens assigned requirement.
2. System validates that the requirement is not locked.
3. Team Member selects the 'Type' field.
4. Team Member chooses either Functional or Non-Functional from predefined values.
5. Team Member clicks 'Save'.
6. System validates selection, stores classification, updates version history, and logs the change.

4. Assign Priority Level

1. Team Member opens assigned requirement.
2. System validates that the requirement is not locked.
3. Team Member selects priority level (Low, Medium, High) from predefined list.
4. Team Member clicks 'Save'.
5. System validates selection, stores priority value, updates version history, and logs the modification.

5. Request Clarification

1. Team Member opens requirement.

2. Team Member clicks 'Request Clarification'.
3. Team Member writes a non-empty clarification question.
4. Team Member submits request.
5. System validates that the requirement exists and message is not empty.
6. System records the clarification request with timestamp and user identity.
7. System stores the request in the discussion thread and notifies Client and Manager.

6. Define Acceptance Criteria

1. Team Member opens assigned requirement.
2. System validates that the requirement is not locked.
3. Team Member navigates to 'Acceptance Criteria' section.
4. Team Member adds one or more non-empty criteria statements.
5. Team Member clicks 'Save'.
6. System validates entries, stores criteria in the database, updates version history, and logs the action.

7. Link Related Requirements

1. Team Member opens requirement.
2. Team Member selects 'Link Requirement'.
3. System displays searchable requirement list.
4. Team Member selects related requirement(s).
5. System validates that linked requirements exist and prevents self-linking.
6. Team Member confirms action.
7. System records the relationship and displays linked requirements in the detail view.

8. Automatic Version History Maintenance

1. Team Member updates any editable requirement field.
2. Team Member clicks 'Save'.
3. System validates that an actual change occurred.
4. System captures the previous state of the requirement.
5. System creates a new version entry.
6. System logs timestamp and editor identity, and preserves the previous version.

9. **Compare Requirement Versions**
    1. Team Member navigates to version history.
    2. System validates that at least two versions exist.
    3. Team Member selects two valid versions.
    4. Team Member clicks 'Compare'.
    5. System highlights differences between selected versions.
    6. System displays comparison data in read-only mode.

b. Non-Functional Requirements

i. Quality Constraints

1. **Performance**
   In normal workload (≤ 20 concurrent users per project), the system shall load the Requirements List page and Requirement Details page within 2 seconds for at least 95% of requests.

2. **Responsiveness**
   The system shall update requirement status changes, comments, and clarification responses within 1 second after submission under standard workload.

3. **Reliability**
   The system shall preserve requirement data and version history with no data loss during normal operation, and shall prevent partial updates.

4. **Traceability**
   The system shall maintain complete version history of each requirement, including editor identity and timestamp, and allow comparison between versions.

5. **Usability**
   The system shall allow users to complete core tasks (create, review, comment, approve) within 3 navigation steps using a clear and consistent interface suitable for students and small teams.

ii. Security Constraints

1. **Authentication**
   The system shall require authenticated login before allowing access to project requirements.

2. **Role-Based Access Control**
   The system shall enforce role-based permissions (Client, Member, Manager) as defined in Phase 2.

3. **Role Integrity**
   The system shall prevent modification of the Client role and ensure that each project always has at least one Manager.

4. **Validation and Safe Handling**
   The system shall validate user inputs for format and length, and handle all stored text safely by escaping output and using parameterized database operations to reduce common injection and scripting risks.

iii. Compatibility & Interface Constraints

1. **Browser Compatibility**
   The system shall function correctly on the latest stable versions of Chrome, Edge, and Firefox.

2. **Responsive Design**
   The system shall support both desktop and mobile screen sizes, as required in the UI design phase.

iv. Technical & External Constraints

1. **Frontend Constraint**
   The system shall be implemented using React as the frontend framework.

2. **Backend Constraint**
   The system shall use NodeJS with Express for backend implementation.

3. **Database Constraint**
   The system shall use MongoDB as the database management system.

4. **Wireframe Tool Constraint**
   The system wireframes shall be designed using Figma and submitted as a wireframe link.

4. Wireframe Development

Figma Link:

https://www.figma.com/design/NsP8x9mWNgFU6T3YU8TLNQ/Untitled?node-id=0-1&t=ZPY055T7Ey6CtXSr-1